Deep-RL using Overcomplete State Representation

Guan-Horng Liu
Robotics Institute Masters
guanhorl@andrew.cmu.edu
Carnegie Mellon University

Sai Prabhakar
Robotics Institute Masters
spandise@andrew.cmu.edu
Carnegie Mellon University

Avinash Siravuru
Mechanical Engineering PhD
avinashs@cmu.edu
Carnegie Mellon University

1 Introduction

Deep learning is a fast growing research area in machine learning with far-reaching applications in diverse domains like speech recognition, image processing, etc. However, extending these ideas to robotic navigation has seen limited success. Recently, the advent of Deep Reinforcement Learning [5, 4] has greatly boosted these efforts and has offered romanticists a promising tool that can operate in the high dimensional observation and action spaces. Moreover, reinforcement learning is a more meaningful way to formulate robotic navigation problems.

Though training a deep network directly end-to-end [2, 9] (instead of separately optimizing each sub-module) is a desirable idea in theory, since we are optimizing the whole system, the two main issues that come arise are (1) poor generalizability since the data-driven system suffers from bias, (2) re-usability since now there are no sub-modules to use elsewhere. For most of the robotic applications, a wide range of measurement sources and high level information is typically used in order to make an intelligent decision. Consider a mobile robot, for example: it's decision-making is based not only on the observable environment states (e.g. laser range sensor, monocular camera), but also other dynamic parameters (e.g. velocity, or kinematic constraint), and even other non-physically abstract information (e.g. destination, behavior planning . . . etc). And to make it even harder, but realistic measurements of the environment are mostly noisy.

The idea of adding reasonable constraints on deep net with respect to different types of input sources has been implemented by Peng. et al. in his mixture of actor-critic experts (MACE) approach[7], where information was split into terrain feature and robot physical states, went through separate networks before combining with each other in the final few stages before output. The concept of network augmentation/modularization has also achieved state-of-the-art results in Visual Question Answering (VQA) recently [1].

Most of the research on DRL uses a complete state representation and tend to avoid data that provide redundant information to reduce the complexity and training time. However, in most of the cases we have additional inputs (image of the robot arm) which give overlapping information, for example camera and lidar. The primary objective of this project is to investigate the effects of using redundant information on my state representation and forming an overcomplete state representation. Specifically, we will be investigating the changes in speed of convergence and the sensitivity to noise of the DRL techniques. We will restrict the experimentation to Deep Deterministic Policy Gradient (DDPG) approach over this project.

2 Background

Deep-Q-Network(DQN) have managed to emulate humans [6] at Atari games using just unprocessed image pixels as input. This is a great testament to their ability to operate in very high-dimensional state spaces with complex and highly dynamic scenarios. However, in all these games, action choices are limited and discrete. It is a different beast altogether when operating in a multi-dimensional and continuous action space.

2.1 Deep Deterministic Policy Gradient

In [10], Silver et al propose a novel *deterministic* policy gradient approach(DPG) ($a = \mu_\theta(s)$) in sharp contrast to the more conventionally used stochastic policy gradient ($\pi_\theta = \mathbb{P}(a/s;\theta)$) algorithms when dealing with continuous action spaces in reinforcement learning problems. They even showed that deterministic policy gradients exist, have a model-free form and follow the gradient of the action-value function. From the practical standpoint, this modification has huge implications. In the stochastic case, the policy gradient integrates over state-action space, whereas in the deterministic case it only integrates over the state space. As a result, computing the stochastic policy gradient may require more samples, especially if the action space has many dimensions.

In order to explore the full state and action space, a stochastic policy is often necessary. To ensure that a deterministic policy gradient based algorithm continues to explore satisfactorily, an off-policy learning algorithm needs to be introduced. In short, choose actions stochastically (to ensure exploration) but learn a deterministic policy gradient. To this end, using the DPG, an off-policy algorithm is developed to estimate the action-value function using a differentiable function approximator. Using the popular Actor-Critic model [11] for off-policy learning and a deep network for function approximation [5] , Lillicrap et al [4] proposed a Deep Deterministic Policy Gradient (DDPG) method to generalize their prior success with discrete action spaces [6] to continuous ones.

2.1.1 Deterministic Policy Gradients

The main objective is to obtain the policy $\pi_\theta(s)$ using reinforcement learning. To this end, we begin by defining a reward function: $R = r_1 + \gamma r_1 + \gamma^2 r_2 ... + \gamma^n r_n$. Here, r_i is reward for the ith time step. A suitable objective function to optimize θ would be the expectation of the total reward:

$$\begin{aligned} L(\theta) &= E[r_1 + \gamma r_2 + \gamma^2 r_3 + ...|\pi_\theta(s)] \\ L(\theta) &= E_{s\sim\ p(s|\theta)}[R] \end{aligned} \tag{1}$$

where the expectations of the total reward R is calculated under some probability distribution $p(s/\theta)$ parameterized by θ. Accordingly, by using the SARSA formula, we can define the Q-function as $Q(s_t, a_t) = R_{t+1}$. Now, we can write the gradient of a deterministic policy $a = \mu(s)$as,

$$\frac{\partial L(\theta)}{\partial \theta} = E_{s\sim\ p(s|\theta)}[\frac{\partial Q}{\partial \theta}] \tag{2}$$

Silver et al [10] proved that this is the policy gradient, i.e. you will get the maximum expected reward as long as you update your model parameters following the gradient formula in (2).

2.1.2 Actor-Critic Model

The Actor-Critic model combines the policy gradient and the value function methods together. Here, the policy function plays the *actor's* role and the value function performs the *critic's* role. Essentially, the actor produces the action a given the current state of the environment s, while the critic produces a signal to critique the actions made by the actor. Here, the Q-learning (SARSA) is the critic model and the policy gradient method serves as an actor model.

In Deep-Q Network, the Q-function is modelled as a neural network $Q^\pi(s, a) \approx Q(s, a, w)$, where w is the weights-vector. Finally, one can formulate the Deep Deterministic Policy Gradient Formula:

$$\frac{\partial L(\theta)}{\partial \theta} = \frac{\partial Q(s, a, w)}{\partial a}\frac{\partial a}{\partial \theta} \tag{3}$$

where, the policy parameters θ can be updated via stochastic gradient ascent. Finally, a loss function is setup to learn the Q-function: $Loss = [r + \gamma Q(s^{'}, a^{'}) - Q(s, a)]^2$. Subsequently, the Q-value can be used to estimate the values of the current actor policy.

2.1.3 Reward Function (*R)*

A potential reward function is the velocity of the car projected along the track direction. The car will try to align itself with the road axis as much as possible in order to maximize this reward. Additional rewards can be added to minimize lateral velocity, minimize deviation from the centre line, etc. This augmentation shows improvement in the learning time [3].

2.1.4 Exploration Strategy

Another key element in this study is the exploration strategy. For this, noise is added using Ornstein-Uhlenbeck process. Ornstein-Uhlenbeck process is a stochastic process which has mean-reverting properties. $dx_t = \psi(\mu - x_t)dt + \sigma dW_t$. Here, ψ means the how "fast" the variable reverts towards to the mean. μ represents the equilibrium or mean value. σ is the degree of volatility of the process.

3 Overcomplete State Representation

Mathematically speaking, overcomplete representations contain more basis vectors than input variables. This can be interpreted as having redundant information. In the context of reinforcement learning, we define it as a representation that not only allows a complete observability of all the relevant state information, but also give additional redundant data about the state. For example using both lidar and camera to navigate. In this example, using just one of them make the state completely observable, i.e. gives us all the relevant information. Hence, using both of them, although gives us additional information about the states, forms an overcomplete state representation for the pertaining task which is navigation.

We have investigated the effects of using overcomplete representation on DRL on two environments, Inverted Pendulum and Car Racing in this project. Environment specific description of overcompleteness for the corresponding environments will be described below.

3.1 Inverted Pendulum

The aim of the inverted pendulum task is to balance a single link pendulum in the upright position. The agent applies torques to achieve this. The agent receives negative rewards proportional to the distance between its current angle and the upright angle and proportional to the magnitude of the angular velocity of the pendulum. The actions an agent outputs are real valued and correspond to the torque in N/m. An optimal policy should swing the pendulum back and forth until it builds up momentum to reach the top. Once upright the policy needs to continue balancing the pendulum.

The environment was implemented using a modified version of its implementation in OpenAI Gym. The original implementation gives the physical parameters– angular position and velocity, of the pendulum. This forms a complete representation of the task considered. The modified version give an image of the pendulum in-addition to the physical parameters forming overcomplete representation.

The pendulum task is a good test task because it can be learned with few samples, and yet the final policy varies a lot according to the state. The torque values required change based on the angle due to gravity. And since the maximum torque that can be applied is not enough to make the pendulum reach upright position from its naturally stable position, the policy must learn to swing the pendulum. It also must switch from building up momentum to removing momentum in order to keep the pendulum balanced. This environment was considered to provide proof of concept. After which, we decided to proceed to more complex environment the car racing game.

3.2 TORCS

TORCS[12] is an open-source racing car simulator capable of simulating sophisticated vehicle dynamics and provides simulated sensor information such as laser range finder, front-view camera image, odometry, etc. Though its environment is limited to racing, and from this point of view one

might argue that other simulators such as GTA-V will give a more realistic simulation for urban driving, from the implementation aspect, we decided to stick with TORCS.

The reward function of TORCS is a function of the velocity and the distance of the car from the central axis of the road. To get maximum reward, the policy learnt must drive the car as fast as possible while remaining as close to the center of the road as possible. The actions that the policy must learn are the steering angle, acceleration and the brake.

TORCS give physical parameter– velocity, position, etc, laser scan and image input. The physical parameters along with the laser scan, form a complete state representation that is considered in this project. Along with an image this complete representation it becomes an overcomplete representation.

3.3 Generalization

It must be noted that there are different ways to form a complete and overcomplete representation using other combination of the inputs available. For example, in the case of the pendulum one of the popular way to form a complete representation is to use four frames of images from time steps *t-3, t-2, t-1,* and t. For the scope of the project we restricted the experimentation to the combination as mentioned above.

4 Experimentation

We used the DDPG algorithm implemented by [8, 3], to avoid having to deal with the implementation details since the aim of the project is only to investigate the effects of overcomplete representation. We describe the hyperparameter and other experimentation details for each of the environments in this section.

4.1 Feature Extraction Modules

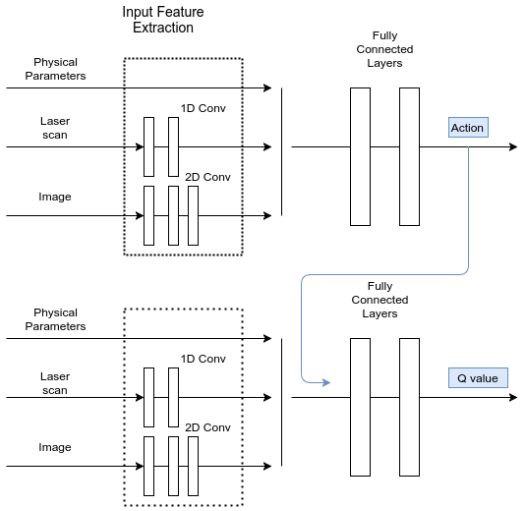

Figure 1: Schematic illustration of the actor and the critic network of DDPG architecture implemented. The top half of the network is the actor and the bottom half is the critic. The feature extraction module takes in input of different modalities like image and laser scan. The extracted feature concatenated are then sent to the remaining layers of actor and critic.

In the original work on DDPG [4, 3], the actor network takes a 1D range-sensor (laser) data, passed through two dense layers, and outputs three bounded continuous actions: steering, acceleration, and brake. However, our network (as shown in in Figure 1) first characterizes different types of sensor inputs and passes them through different feature extraction networks. The outputs of each feature extraction network are later merged and passed through the standard Q-network ($Q(s, a, w)$). This uses our domain knowledge of the inputs we expect from the environment, to efficiently extract the required features from the input. Using this setup eliminates the need for large fully connected layers,

and hence lesser number of parameters to train, this reduces the training time. This technique, can be seen from literature improves the quality of features learnt.

4.2 Inverted Pendulum Environment

We conducted two base experiments one with complete state representation– using only physical parameters. The other with overcomplete state representation– using physical parameter and image.

The overall network architecture for both the representations were kept constant as shown in Figure 1. The only difference between them is in the feature extraction module. In the complete representation, since the image and laser inputs are absent those feature extraction sub-modules are ignored. Similarly in the overcomplete representation laser feature extraction sub-module is ignored.

Layers	Laser Feature Sub-module	Image Feature Sub-module (TORCS)	Image Feature Sub-module (Pendulum)
Input dimension	19*1	64*64*3	64*64*1
Hidden Layer 1	4, 4*1 Conv. layers	16, 4*4 Conv. layers, stride = 4	8, 4*4 Conv. layers, stride = 4
Hidden Layer 2	4, 2*1 Conv. layers	32, 2*2 Conv. layers, stride = 2	8, 2*2 Conv. layers, stride = 2
Hidden Layer 3	None	32, 2*2 Conv. layers, stride = 2	8, 2*2 Conv. layers, stride = 2

Table 1: Configuration of feature extraction module. It should be noted that to increase the training speed, batch normalization was performed after each layer in the module.

The image feature extraction module's configuration is shown in Table 1. In addition to the configuration shown in the table to increase the training speed batch normalization was performed after each layer in the module.

The actor network had three hidden fully connected layers using ReLU activations. Each layer had 16 units. The actor network output was a fully connected layer with one unit and a linear activation. The critic network had three hidden, fully connected layers using ReLU activations. Each layer had 32 units. The critic output was a fully connected layer with one unit using linear activation.

A replay memory of size 200 was used with a warm-up period of 100 iterations. Exploration was done using noise collected from Ornstein-Uhlenbeck process. This process is essentially a Gaussian distribution with added momentum simulating the Brownian motion. The soft target network update rate was 1×10^{-2}. Training used a batch size of 32 and the RMSProp optimizer with a base learning rate of .001, with gamma value of 0.99.

4.3 TORCS

Similar to the pendulum environment, we conducted two base experiments one with complete state representation– using only physical parameters and laser scan. The other with overcomplete state representation– using physical parameter, laser scan and image. The overall network architecture for both the representations were kept constant as shown in Figure 1. The only difference between them is in the feature extraction module.

The image and laser scan's feature extraction module's configuration is shown in Table 1. In addition to the configuration shown in the table to increase the training speed batch normalization was performed after each layer in the modules. Both the actor and critic network had two hidden layers of 200 units each with ReLu activation. The output layer uses linear activation with actor have three, and the critic using one hidden unit.

A replay memory of size 50,000 was used with a warm-up period of 10,000 iterations. Exploration was done using noise collected from Ornstein-Uhlenbeck process. The soft target network update rate was 1×10^{-2}. Training used a batch size of 64 and the RMSProp optimizer with a base learning rate of .001, with gamma value of 0.99.

It should be noted that the network configuration described in this section was selected after a series of experiments and intuition. We selected the best architecture which gives the best performance without using lots of parameters and hence increasing the training period.

4.4 Convergence

For the first set of experiment, the DDPG architecture as described above was trained on both pendulum and the TORCS, using both complete and overcomplete representation. The networks were trained for a fixed number of iterations. The results for pendulum and TORCS are shown in Figure 2 and 3. The plots represent the best training instance among all the experiments.

From the image we can see that using training time decreases the reward for the pendulum as expected, but increases for TORCS when using overcomplete representation. The increase in training time for TORCS could be because of the high increase in the number of parameters to be trained when using images.

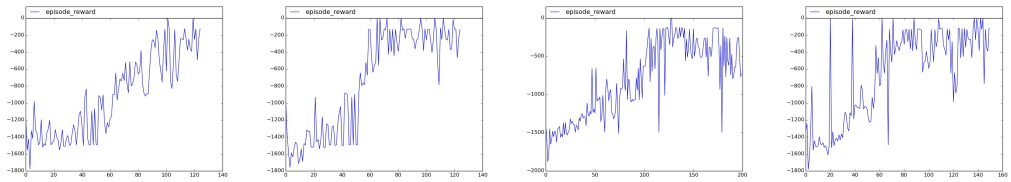

Figure 2: Pendulum: Rewards received during training using (Far Left) only physical parameters, and (Left) using both physical parameter and image. Rewards received during training with 10% uniform guassian noise added to physical parameter, using (Right) only physical parameters, and (Far Right) using both physical parameter and image. The plots represent the best training instance among all the experiments.

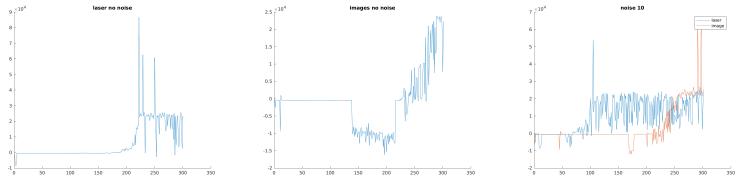

Figure 3: TORCS: Rewards received during training using (Left) physical parameters and laser scan, and (Center) using physical parameter, laser scan and image. (Right) Rewards received during training with 10% uniform guassian noise added to laser scan. The plots represent the best training instance among all the experiments.

4.5 Robustness

For the second set of experiments we tested the networks' sensitivity no noise. In the case of Pendulum a 10% guassian noise was added to the physical parameters and for TORCS the noise was added to the laser scan. The results for the pendulum and TORCS are shown in Figure 2 and 3. The plots represent the best training instance among all the experiments. We observe an increased robustness to noise in both the environments when using the overcomplete representation.

5 Results and Conclusion

Test time rewards are shown in the Table 2. The test results also confirm the increased robustness to noise while using overcomplete representation. It should be noted that in the absence of the noise, the performance of overcomplete representation is slightly worse that the complete representation in the case of the pendulum, in the case of TORCS it is slightly better. This could be because the overcomplete representation gives best results when the policy to learn is complex.

	Complete representation (TORCS)	Overcomplete representation (TORCS)	Complete representation (Pendulum)	Overcomplete representation (Pendulum)
Without noise	$2.3 * 10^4$	$2.3 * 10^4$	-174	-199
With noise	$2.0 * 10^4$	$2.8 * 10^4$	-250	-183

Table 2: Average reward received during testing for both pendulum and TORCS.

Hence we conclude that overcomplete representation increases the performance and the robustness to noise in deep reinforcement learning techniques. The improvement is more noticeable in complex environments.

6 Future work

More experimentation with other combinations of inputs as described in Section 3.3, needs to be performed. This is in order to confirm the advantage of using overcomplete representation. We also would like to experiment on performance variation in the network when we completely remove a sensor input in both the cases. These experiments will bring a more complete understanding of the advantages and disadvantages of using overcomplete representation.

References

[1] Jacob Andreas, Marcus Rohrbach, Trevor Darrell, and Dan Klein. Deep compositional question answering with neural module networks. *CoRR*, abs/1511.02799, 2015.

[2] Mariusz Bojarski, Davide Del Testa, Daniel Dworakowski, Bernhard Firner, Beat Flepp, Prasoon Goyal, Lawrence D. Jackel, Mathew Monfort, Urs Muller, Jiakai Zhang, Xin Zhang, Jake Zhao, and Karol Zieba. End to end learning for self-driving cars. *CoRR*, abs/1604.07316, 2016.

[3] Yan-Pan Lau. Using keras and deep deterministic policy gradient to play torcs. `https://yanpanlau.github.io/2016/10/11/Torcs-Keras.html`, 2016.

[4] Timothy P. Lillicrap, Jonathan J. Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. Continuous control with deep reinforcement learning. *CoRR*, abs/1509.02971, 2015.

[5] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. Playing atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602*, 2013.

[6] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.

[7] Xue Bin Peng, Glen Berseth, and Michiel van de Panne. Terrain-adaptive locomotion skills using deep reinforcement learning. *ACM Transactions on Graphics (Proc. SIGGRAPH 2016)*, 35(4), 2016.

[8] Matthias Plappert. Deep reinforcement learning for keras. `https://github.com/matthiasplappert/keras-rl`, 2016.

[9] Craig Quiter. Deepdrive: self-driving car ai, 2016.

[10] David Silver, Guy Lever, Nicolas Heess, Thomas Degris, Daan Wierstra, and Martin Riedmiller. Deterministic policy gradient algorithms. In *ICML*, 2014.

[11] Richard S Sutton and Andrew G Barto. *Reinforcement learning: An introduction*, volume 1. MIT press Cambridge, 1998.

[12] Bernhard Wymann, E Espié, C Guionneau, C Dimitrakakis, R Coulom, and A Sumner. Torcs, the open racing car simulator. *Software available at http://torcs. sourceforge. net*, 2000.